

Reflected Cross-Site Scripting (XSS) — Vulnerability Report

1. Executive Summary

A **Reflected Cross-Site Scripting (XSS)** vulnerability was identified in the target web application.

The application reflects user-controlled input directly into the HTML response without applying appropriate output encoding or sanitization. Because the supplied input is interpreted as HTML/JavaScript by the browser, an attacker can inject a malicious client-side script into the application's response.

This vulnerability was reproduced in the PortSwigger Web Security Academy lab:

Lab: Reflected XSS into HTML context with nothing encoded

Testing Tool: Burp Suite Professional

Vulnerability: Reflected Cross-Site Scripting (XSS)

Severity: Medium

2. Vulnerability Details

Field	Details
Vulnerability	Reflected Cross-Site Scripting
CWE	CWE-79 — Improper Neutralization of Input During Web Page Generation
OWASP Category	A03: Injection
XSS Type	Reflected XSS
Injection Context	HTML context

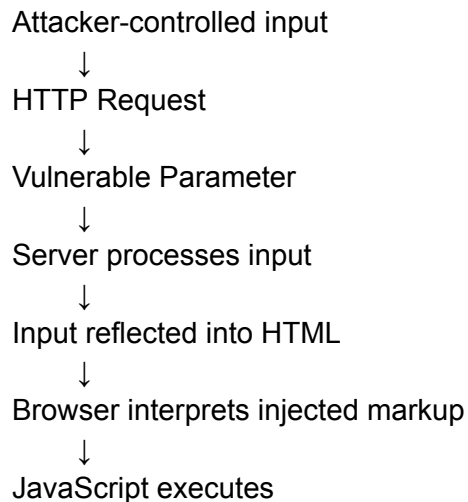
Input Encoding	None / insufficient
User Interaction	Victim must access the crafted URL/request
Impact	JavaScript execution in the victim's browser

3. Affected Functionality

The vulnerable functionality accepts a user-controlled parameter through an HTTP request and reflects that parameter into the application's HTML response.

The application does not properly encode the supplied value before placing it into the HTML document.

Data Flow



4. Proof of Concept

Using **Burp Suite Professional**, the HTTP request was intercepted and the vulnerable parameter was identified.

A harmless XSS proof-of-concept payload was inserted into the affected parameter.

Example:

```
<script>alert(document.domain)</script>
```

The server reflected the supplied input into the HTML response without encoding the HTML characters.

As a result, the browser interpreted the injected `<script>` element as executable JavaScript.

Expected Result

The browser executes the supplied JavaScript and displays the proof-of-concept alert.

JavaScript executed successfully



XSS confirmed

5. Burp Suite Testing Procedure

Step 1 — Open the Lab

Open the PortSwigger lab and navigate to the functionality containing the search/input parameter.

Step 2 — Intercept the Request

Enable **Burp Proxy** → **Intercept** and submit a normal value through the vulnerable functionality.

Example:

test

Step 3 — Inspect the Request

Send the intercepted request to **Repeater**.

In Burp Repeater, identify the parameter containing the user-controlled input.

Step 4 — Test for Reflection

Replace the normal input with a harmless XSS test payload:

```
<script>alert(document.domain)</script>
```

Send the request.

Step 5 — Inspect the Response

The response contains the supplied input directly within the HTML response.

The important observation is that characters such as:

```
<  
>  
"  
,
```

are not appropriately encoded before being returned to the browser.

Step 6 — Verify Execution

When the resulting page is rendered by the browser, the injected JavaScript executes.

This confirms that the input is not merely reflected but is interpreted as executable HTML/JavaScript.

6. Root Cause

The primary root cause is **improper handling of untrusted user input before inserting it into an HTML response**.

The application effectively performs:

User Input → HTML Response

instead of safely performing contextual output encoding:

User Input → HTML Encoding → HTML Response

Because the input is inserted into an HTML context without proper encoding, attacker-controlled markup can become part of the document structure.

7. Security Impact

Successful exploitation of reflected XSS may allow an attacker to execute arbitrary JavaScript in the security context of the vulnerable website.

Depending on the application's functionality and browser security controls, potential consequences include:

- Execution of attacker-controlled JavaScript
- Modification of page content
- Performing actions on behalf of the victim
- Theft of accessible client-side data
- Phishing through manipulated page content
- Session-related attacks where protections are insufficient
- Redirecting users to attacker-controlled content
- Compromising the trust relationship between the user and the website

The actual impact depends on the application's authentication, session-management, and browser security controls.

8. Severity Assessment

Severity: Medium

The vulnerability allows attacker-controlled JavaScript to execute in a victim's browser, but exploitation generally requires the victim to interact with a crafted URL or request.

The risk may become **High** in applications containing sensitive authenticated functionality or where successful exploitation can access valuable client-side information.

9. Remediation

1. Context-Aware Output Encoding

User-controlled data should be properly encoded according to its output context.

For HTML body content, HTML entity encoding should be applied.

For example:

< → <
> → >
" → "
' → '

2. Avoid Unsafe HTML Injection

Do not directly insert untrusted input into HTML using unsafe DOM APIs such as:

```
element.innerHTML = userInput;
```

Prefer safer alternatives such as:

```
element.textContent = userInput;
```

3. Input Validation

Apply appropriate server-side validation where practical.

Input validation should be considered an additional security layer rather than the primary XSS defense.

4. Content Security Policy

Implement a strong **Content Security Policy (CSP)** to reduce the impact of XSS vulnerabilities.

CSP should be treated as defense-in-depth and should not replace proper output encoding.

5. Security Testing

Test all locations where user-controlled data is reflected into:

- HTML
- HTML attributes
- JavaScript
- CSS
- URLs
- DOM operations

10. Evidence

Request

GET /?search=<XSS-PAYLOAD> HTTP/1.1
Host: vulnerable-lab

Payload

<script>alert(document.domain)</script>

Response Behavior

The application reflects the supplied payload into the HTML response without appropriate encoding.

Result

Input Reflected → HTML Interpreted → JavaScript Executed

Conclusion: Reflected XSS confirmed.

11. Tools Used

Tool	Purpose
Burp Suite Professional	HTTP interception, request modification and response analysis
Web Browser	Rendering response and validating JavaScript execution
PortSwigger Web Security Academy	Controlled vulnerable lab environment

12. Final Conclusion

The tested application is vulnerable to **Reflected Cross-Site Scripting (XSS)** because attacker-controlled input is reflected into an HTML context without appropriate output encoding.

The vulnerability was successfully reproduced using **Burp Suite Professional** in the PortSwigger controlled lab environment.

Implementing **context-aware output encoding**, avoiding unsafe HTML manipulation, applying appropriate input validation, and deploying a restrictive CSP can significantly reduce the risk of XSS exploitation.